

Scientific Programming with Python

Assignment: Unit Test: DNA Transcription

Karl N. Kirschner

Department of Computer Science, University of Applied Sciences Bonn-Rhein-Sieg,
Sankt Augustin, Germany

June 1, 2026

Goal The goal of this assignment is to apply unit test verification ideas to DNA[1] sequence processing. You will also generate toy data that generates differing DNA sequences that could be used for future algorithm development. (The generation of artificial/synthetic data – i.e., toy data – that can be used for algorithm development — e.g. visualization code – and data exploration is an important concept in scientific research [2].)

Problem and Input Data In life, DNA is transcribed to RNA for gene expression (e.g., creating proteins: DNA → RNA → protein) [1, 3]. DNA is composed of the following four nucleotide bases: A (adenine), T (thymine), C (cytosine) and G (guanine). DNA is transcribed into RNA by replacing the nucleotide base T with U (uracil).

In bioinformatics, incorrect or corrupted data can lead to wrong scientific conclusions or medical misdiagnosis. If a script improperly handles missing data or incorrectly manipulates existing data, the downstream analysis is useless. To prevent this, data scientists write unit tests to guarantee that their functions behave exactly as expected before processing critical.

Assignment Tasks

Task 1: Transcription Function Create a user-defined function that transcribes a given DNA string (e.g., "ATCGAT") to its RNA equivalent ("AUCGAU").

Hint: `str.replace()`

Task 2: Unit Test Write a unit test that verify:

- A standard sequence transcribes correctly.
- That case insensitivity works (e.g., "atcg" becomes "AUCG").
- A `ValueError` is raised when an invalid character (like "X") is passed.

Hint: If using the `unittest` library, for the `ValueError` test, the use of a `try-except` statement with an `assertTrue` is one possible approach.

Task 3: Toy Data Create a user-defined function that randomly generates and return a single DNA sequence string composed of the nucleotides 'A', 'T', 'C', and 'G'. The length of the generated string should also be randomly assigned (e.g., between 10 and 100 characters).

Hint: the `random` library

Allowed Python3 [4, 5] functions & libraries/modules

- all built-in functions
- pytest or unittest
- random library
- typing library (if needed)

Assignment Due Turn in your solution as a **Jupyter-notebook** [6] or as Python scripts (e.g., filename.py) to **LEA** by **Monday, June 8th, 2026 at 13:30**.

Note 1 : Please include your **SciPro_ID** at the **top** of your notebook.

Note 2 : **Do not** consider **significant figures** in this solution.

References

- [1] Minchin, S., and Lodge, J. (2019) Understanding biochemistry: Structure and Function of Nucleic Acids. Essays in Biochemistry, 63(4), 433.
- [2] Stickland, R., Mohammad M., Zormpa, E., Santos, L. and AIM-RSF team (2025) Demystifying Synthetic Data, <https://github.com/aim-rsf/Synthetic-Data.git>. Accessed on May 30, 2026
- [3] Gahtan, V., Olson, E.T. and Sumpio, B.E. (2002) Molecular Biology: A Brief Overview. Journal of vascular surgery, 35(3), pp.563-568.
- [4] Python Software Foundation. Python Language Reference, version 3.11. Available at <http://www.python.org>. WAccessed on October 22, 2023.
- [5] van Rossum, G. Python tutorial, Technical Report CS-R9526, Centrum voor Wiskunde en Informatica (CWI), Amsterdam, 1995.
- [6] Kluyver, T. et al., (2016) Jupyter Notebooks – a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt, eds. Positioning and Power in Academic Publishing: Players, Agents and Agendas. pp. 87–90.